LOW-PASS FILTER DESIGN FOR ULTRASONIC DISTANCE MEASUREMENT

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....4
Recommended Citation.....4

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Design a first-order discrete-time low-pass filter for a chosen cut-off frequency and sample time, and implement it in Simulink.	☐
2	Measure the noise reduction and the lag introduced by the filter for several cut-off frequencies, using ultrasonic distance measurements.	☐
3	Select a cut-off frequency that balances smoothing against delay for use inside a feedback loop, and explain the choice.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware set-up described in the practical exercise, Ultrasonic Sensor, which details how the ultrasonic sensor is connected to the Arduino Uno and how distance is read into Simulink. The set-up is not repeated here.

A flat piece of card, at least 10 cm square, is used as the target, and a ruler or two marks on the bench at 20 cm and 40 cm from the sensor are used to move it between known positions.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. Ultrasonic distance measurements contain noise and occasional false readings. A first-order discrete-time low-pass filter is designed for three cut-off frequencies, and the noise and lag of each are measured, so that the trade-off between them can be judged. The exercise applies the ACE-CORE learning guide, Discrete-Time Low-Pass Filter Design, to a second sensor after the DC motor.

Background: The First-Order Discrete-Time Filter

A first-order low-pass filter with cut-off frequency f_c has the time constant $\tau_f = 1/(2\pi f_c)$. Discretised with a zero-order hold at sample time T_s, it becomes the difference equation:

$$y[k] = \alpha\, y[k-1] + (1-\alpha)\, x[k], \qquad \alpha = e^{-T_s/\tau_f} = e^{-2\pi f_c T_s}$$

where x is the raw distance and y the filtered distance. A value of α close to 1 gives strong smoothing but a slow response. The filter responds to a step with a time constant τ_f, reaching 63.2% of the step after one time constant. The values for the three cut-off frequencies used in this exercise, at $T_s = 0.05$ s, are given in Table 2.

Table 2: *Filter Coefficients at a Sample Time of 0.05 s*

Cut-off frequency, f_c (Hz)	Time constant, τ_f (s)	α	$1-\alpha$
5	0.032	0.208	0.792
1	0.159	0.730	0.270
0.2	0.796	0.939	0.061

MATLAB Script: calculating the filter coefficient and checking it with c2d

```
Ts = 0.05;                      % sample time [s]
fc = 1;                         % cut-off frequency [Hz]
tau_f = 1/(2*pi*fc);            % filter time constant [s]
alpha = exp(-Ts/tau_f)          % coefficient on y[k-1]
Hd = c2d(tf(1,[tau_f 1]), Ts, 'zoh')   % check: Hd = (1-alpha)z^-1/(1-alpha z^-1)
```

Note that the zero-order-hold form from `c2d` multiplies $x[k-1]$ rather than $x[k]$, which adds one sample of delay. Either form can be used. The form in the equation above responds one sample sooner.

Measure the Noise and the Lag

[Screenshot to insert] *Ultrasonic sensor block (distance, cm) → Discrete Transfer Fcn or MATLAB Function (low-pass filter) → Scope; raw and filtered distance logged as x and y.*

Figure 1: *Simulink Model for Filtering the Ultrasonic Distance*

1. Add the filter to the ultrasonic model from the practical exercise, Ultrasonic Sensor, using a MATLAB Function block or a Discrete Transfer Fcn block with numerator $[1-\alpha\ 0]$ and denominator $[1\ -\alpha]$. Set $T_s = 0.05$ s and $f_c = 5$ Hz.
2. Log the raw distance x, the filtered distance y and the time t to the workspace.
3. Hold the card still at 20 cm for 5 s, then move it quickly to 40 cm and hold it there for 5 s.

4. Run the script below to calculate the noise (the standard deviation while the card is still) and the lag (the time for the filtered signal to reach 63.2% of the step).
5. Repeat for f_c = 1 Hz and f_c = 0.2 Hz, and record the results in Table 3.

MATLAB Script: measuring the noise and lag of the filtered distance

```
x_k = x.signals.values(:);              % raw distance [cm]
y_k = y.signals.values(:);              % filtered distance [cm]
t_k = t.signals.values(:);              % time [s]
still = t_k > 1 & t_k < 4.5;                    % card held at 20 cm
noise_raw = std(x_k(still))                     % [cm]
noise_flt = std(y_k(still))                     % [cm]

t_step = t_k(find(x_k > 30, 1));                % raw signal crosses halfway
d0 = mean(y_k(still));  d1 = mean(y_k(t_k > t_k(end)-2));
t63 = t_k(find(y_k >= d0 + 0.632*(d1-d0) & t_k >= t_step, 1));
lag = t63 - t_step                              % approximate filter lag [s]
plot(t_k, x_k, 'Color', [0.6 0.6 0.6]); hold on; plot(t_k, y_k, 'b');
grid on; xlabel('Time [s]'); ylabel('Distance [cm]'); legend('Raw','Filtered');
```

Simulated example results are shown in Figure 2, using a raw signal with 0.5 cm of random noise and four false readings. They show the shape to expect rather than measured data. At 5 Hz the filter follows the step closely but passes most of the noise and spikes. At 0.2 Hz the signal is very smooth, but takes almost a second to reach the new distance.

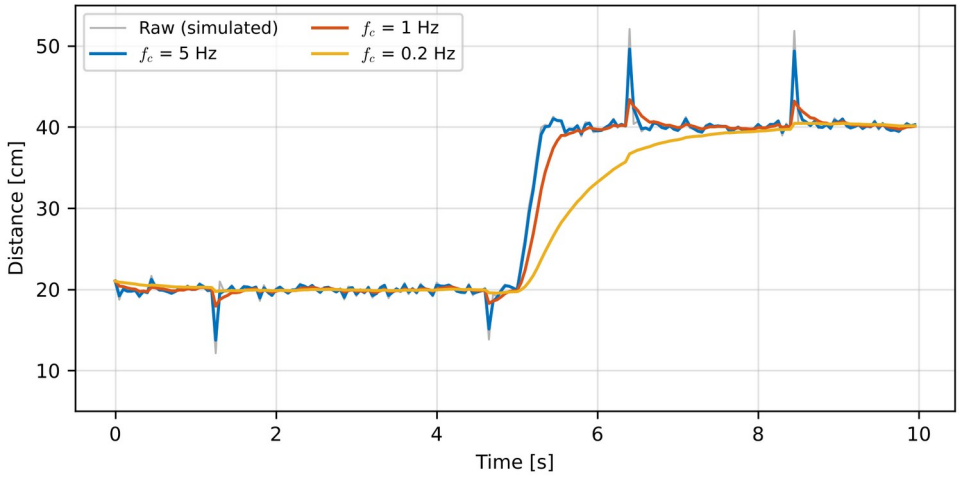


Figure 2: *Simulated Example of Raw and Filtered Distance for a Step from 20 cm to 40 cm*

Table 3: *Noise and Lag Results for Your Sensor*

f_c (Hz)	Noise, raw (cm)	Noise, filtered (cm)	Measured lag (s)	Expected lag, τ_f (s)
5				0.032
1				0.159
0.2				0.796

Select a Filter

Plot the filtered noise against the measured lag for the three filters. Then answer the following questions.

1. The filtered distance will be used by a controller that holds an object at a fixed distance from the sensor. Which cut-off frequency would you choose, and why?
2. Explain why adding lag inside a feedback loop makes the loop harder to stabilise.
3. The low-pass filter smears the false readings rather than removing them. Would a median filter over the last three samples handle them better? Explain why.
4. How does α change if the sample time is halved but f_c is kept the same?

4.0 Concluding Remarks

This practical exercise designed a first-order discrete-time low-pass filter for ultrasonic distance measurements. Using MATLAB, Simulink and ACE-Lab hardware, the filter coefficient was calculated from the cut-off frequency and sample time, and the noise and lag of three filters were measured.

Lower cut-off frequencies reduced the noise but increased the lag, with the measured lag close to the filter time constant. A filter used inside a feedback loop must therefore be chosen with the whole loop in mind: enough smoothing to give a usable measurement, without adding so much delay that the controller becomes slow or unstable.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Low-Pass Filter Design for Ultrasonic Distance Measurement. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://zenodo.org/records/23725316